Structured Inquiry

Create a maven project with all dependencies required for the application in CI/CD pipeline.

Section 1:

Step 1: Java has been already installed check the version

Step 2: Install Maven using below Command

CMD : sudo apt update
sudo apt install maven -y

Step 3: Create Maven Project
run the below command

CMD:
mvn archetype:generate -DgroupId=com.example.app \
-DartifactId=demo-app \
-DarchetypeArtifactId=maven-archetype-quickstart \
-DinteractiveMode=false

This will create a project with name demo-app

Step 4: change the directory to demo-app using below command

CMD : cd demo-app

Step 5: list all the files from the demo-app directory

CMD : ls
o/p: pom.xml , src

Step 6: Understand the project structure

```
demo-app/
├── pom.xml
├── src/
│   ├── main/java
│   └── test/java
```

Step 7: edit the pom.xml

CMD: nano pom.xml

add the below sections to pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example.app</groupId>
  <artifactId>demo-app</artifactId>
  <packaging>jar</packaging>
  <version>1.0-SNAPSHOT</version>
  <name>demo-app</name>
  <url>http://maven.apache.org</url>
  <dependencies>
    <dependency>
      <groupId>ch.qos.logback</groupId>
      <artifactId>logback-classic</artifactId>
      <version>1.4.14</version>
    </dependency>

    <!-- JUnit for testing -->
    <dependency>
        <groupId>junit</groupId>
        <artifactId>junit</artifactId>
        <version>4.13.2</version>
        <scope>test</scope>
    </dependency>

    <!-- Logging -->
    <dependency>
        <groupId>org.slf4j</groupId>
        <artifactId>slf4j-api</artifactId>
        <version>2.0.9</version>
    </dependency>

    <!-- Apache Commons -->
    <dependency>
        <groupId>org.apache.commons</groupId>
        <artifactId>commons-lang3</artifactId>
        <version>3.14.0</version>
    </dependency>


  </dependencies>
<build>
<plugins>

        <!-- Compiler Plugin -->
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <version>3.11.0</version>
```

```
            <configuration>
                <source>17</source>
                <target>17</target>
            </configuration>
        </plugin>

        <!-- Test Plugin -->
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-surefire-plugin</artifactId>
            <version>3.1.2</version>
        </plugin>

    </plugins>
</build>
</project>
```

save and exit nano using **ctrl+O, Enter** and **ctrl+X**

Step 8: Write the java application code

CMD: nano src/main/java/com/example/app/App.java

copy the below code to App.java

```
package com.example.app;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.apache.commons.lang3.StringUtils;

public class App {

    private static final Logger logger = LoggerFactory.getLogger(App.class);

    public static void main(String[] args) {
        String name = "CI/CD Pipeline";

        if (StringUtils.isNotBlank(name)) {
            String message = greet(name);
            logger.info(message);
            System.out.println(message);
        } else {
            logger.error("Name is empty!");
        }
    }

    public static String greet(String name) {
        return "Hello, " + name + "! Welcome to Maven CI/CD Demo.";
    }
```

}

Step 9: Write test code

CMD: nano src/test/java/com/example/app/AppTest.java

copy the below code to AppTest.java

```
package com.example.app;

import org.junit.Test;
import static org.junit.Assert.assertEquals;

public class AppTest {

    @Test
    public void testGreet() {
        String result = App.greet("Student");
        assertEquals("Hello, Student! Welcome to Maven CI/CD Demo.", result);
    }
}
```

Step 10: Build the project

CMD: mvn clean install

Step 11: Run the application

CMD: mvn exec:java -Dexec.mainClass="com.example.app.App"

```
[INFO] ----------------------< com.example.app:demo-app >----------------------
[INFO] Building demo-app 1.0-SNAPSHOT
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- exec-maven-plugin:3.6.3:java (default-cli) @ demo-app ---
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/resolver/maven-resolver-util/1.9.24/maven-resolver-util-1.9.24.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/resolver/maven-resolver-util/1.9.24/maven-resolver-util-1.9.24.pom (2.2 kB at 30 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/resolver/maven-resolver/1.9.24/maven-resolver-1.9.24.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/resolver/maven-resolver/1.9.24/maven-resolver-1.9.24.pom (25 kB at 295 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/resolver/maven-resolver-api/1.9.24/maven-resolver-api-1.9.24.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/resolver/maven-resolver-api/1.9.24/maven-resolver-api-1.9.24.pom (2.2 kB at 28 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-exec/1.6.0/commons-exec-1.6.0.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-exec/1.6.0/commons-exec-1.6.0.pom (11 kB at 135 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-parent/93/commons-parent-93.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-parent/93/commons-parent-93.pom (79 kB at 847 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/junit/junit-bom/5.14.1/junit-bom-5.14.1.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/junit/junit-bom/5.14.1/junit-bom-5.14.1.pom (5.7 kB at 67 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm/9.9.1/asm-9.9.1.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm/9.9.1/asm-9.9.1.pom (2.4 kB at 28 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm-commons/9.9.1/asm-commons-9.9.1.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm-commons/9.9.1/asm-commons-9.9.1.pom (2.8 kB at 18 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm-tree/9.9.1/asm-tree-9.9.1.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm-tree/9.9.1/asm-tree-9.9.1.pom (2.6 kB at 28 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/resolver/maven-resolver-util/1.9.24/maven-resolver-util-1.9.24.jar
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/resolver/maven-resolver-api/1.9.24/maven-resolver-api-1.9.24.jar
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-exec/1.6.0/commons-exec-1.6.0.jar
Downloading from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm-commons/9.9.1/asm-commons-9.9.1.jar
Downloading from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm/9.9.1/asm-9.9.1.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/resolver/maven-resolver-util/1.9.24/maven-resolver-util-1.9.24.jar (196 kB at 2.0 MB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm-tree/9.9.1/asm-tree-9.9.1.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm-tree/9.9.1/asm-tree-9.9.1.jar (52 kB at 304 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm-commons/9.9.1/asm-commons-9.9.1.jar (75 kB at 436 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-exec/1.6.0/commons-exec-1.6.0.jar (69 kB at 389 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm/9.9.1/asm-9.9.1.jar (126 kB at 694 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/resolver/maven-resolver-api/1.9.24/maven-resolver-api-1.9.24.jar (157 kB at 836 kB/s)
15:44:03.836 [com.example.app.App.main()] INFO com.example.app.App -- Hello, CI/CD Pipeline! Welcome to Maven CI/CD Demo.
Hello, CI/CD Pipeline! Welcome to Maven CI/CD Demo.
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  2.273 s
[INFO] Finished at: 2026-04-08T15:44:03+05:30
[INFO] ------------------------------------------------------------------------
```

output:

Hello, CI/CD Pipeline! Welcome to Maven CI/CD Demo.
Hello, CI/CD Pipeline! Welcome to Maven CI/CD Demo.

Section 2:

Push the above demo-app directory to github by creating new repo, create a Jenkinsfile and add the below code to Jenkinsfile

```
pipeline {
    agent any

    tools {
        maven 'Maven3'
        jdk 'JDK17'
    }

    stages {

        stage('Checkout') {
            steps {
                git 'https://github.com/Naveen04jan/demo-app.git'
            }
        }

        stage('Build') {
            steps {
                sh 'mvn clean compile'
            }
        }

        stage('Test') {
            steps {
                sh 'mvn test'
            }
        }

        stage('Package') {
            steps {
                sh 'mvn package'
            }
        }
    }
}
```

Steps to run jenkins: Follow the same procedure for running the jenkins server from experiment 3 and 5 create the new pipeline in jenkins and build the pipeline.